

Jev Clearly Explained: I Built a Decision Layer for an AI Agent

A hands-on test of Jev's typed decisions, probability outputs, latency, and failure modes inside an agent-style workflow.



YOUSSEF HOSNI

SEP 26, 2026  PAID

Get 50% off for 1 year

AI agents often use general-purpose LLMs for decisions that do not require generated text, such as routing a request, judging tool risk, or deciding whether human approval is needed.

In this blog, I explore **Jev**, a decision model designed for bounded semantic judgments with typed outputs and probability distributions. I build a tool-call risk gate around an AI agent, where Jev evaluates risk, destructiveness, and approval requirements while ordinary Python code retains control over whether an action is allowed, reviewed, or blocked.

I then compare Jev with GPT-5.6 Terra on the same decision workflow across 40 measured calls per model. In this experiment, Jev reached a median latency of **242.6 ms** versus **1,511.5 ms** for Terra, while the estimated API cost was roughly **82× lower**. More importantly, the experiments show where this interface is useful—and where deterministic code or a general-purpose reasoning model remains the better choice.

50% Discount on all my books, courses &
bundles for To Data & Beyond paid
subscribers

Upgrade to paid

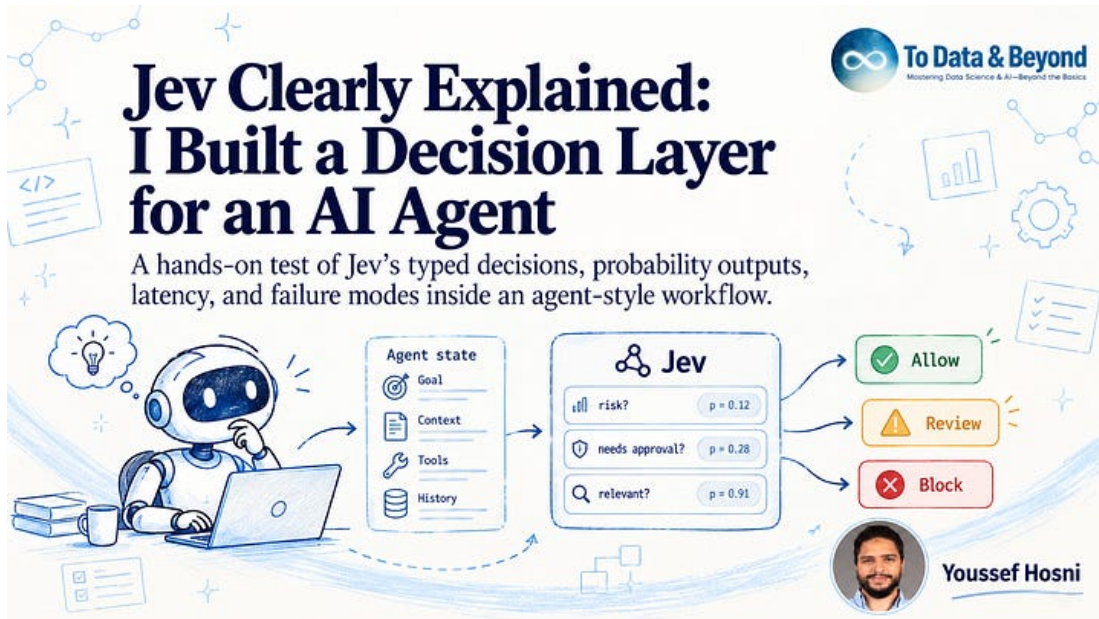


Table of Contents:

1. **Why an Agent Can Waste LLM Calls on Decisions That Need No Text**
2. **How Jev Turns Semantic Judgment into a Software Primitive**
3. **Building a Jev Tool-Call Risk Gate**
4. **Jev vs. GPT-5.6 Terra on the Same Decisions**
5. **Where Jev Fits, and Where a Normal if Statement or LLM Still Wins**

My new live cohort: [Claude Agent Engineering](#). Six 3-hour live sessions, six office hours, and one agent system you build layer by layer. Only 20 seats, starting 1 November.

Founding rate: **\$320** instead of **\$400** until Sunday, 27 September. Use code **CLAUDECOURSE20**.

35% discount for paid subscribers

Upgrade to paid

Claude Agent Engineering Live Course



Claude Agent Engineering
A 6-Week Live Cohort

Claude Code · Claude Cowork · Claude Skills · Claude Agent SDK · Agent Loops · Context Engineering

Live workshops · Weekly office hours · Private Slack community

\$400

The banner features a central diagram of the Claude Agent architecture. At the top, a code editor shows a function `main() { agent = new ClaudeAgent({ tools, memory, skills }); result = await agent.run(); return result; }`. Below this, the 'CLAUDE AGENT' is shown as a central hub. To its left are 'SKILLS' (Search, Code, Docs, Compute, Integrations) and 'AGENT WORKFLOW' (Plan, Act, Evaluate). To its right is the 'AGENT LOOP' (Observe, Act, Reflect, Improve). At the bottom is 'CONTEXT ENGINEERING' (User intent & Goals, Knowledge & Context, Tools & Environment).

1. Why an Agent Can Waste LLM Calls on Decisions That Need No Text

Consider a coding agent that has just inspected a repository and is deciding what to do next. It may need to choose between reading another file, modifying the implementation, running the tests, or stopping because the task appears complete. Later in the same run, it may need to judge whether a shell command is safe, whether a test failure is relevant, or whether the result is good enough to return to the user.

These are meaningful decisions, but many of them do not require generated language. The application already knows the possible actions. What it needs from the model is a judgment over those actions.

Modern agent harnesses usually give this job to the same general-purpose LLM that handles the rest of the task. LangChain, for example, describes the core agent loop as a model repeatedly deciding what to do, executing a tool, observing the result, and continuing until the task is complete. The surrounding harness adds state, tool execution, permissions, context management, and other controls around that loop.

Tool calling and structured outputs made this architecture much easier to build. Instead of asking a model to produce prose such as “I think we should run the tests now,” the application can require something closer to:

```
action = "run_tests"
```

Or it can constrain the result to a predefined JSON schema. This removes much of the brittle parsing that early LLM applications needed, but it does not change the basic inference process. A normal autoregressive LLM still produces an output sequence token by token, even when the application only needs to choose one value from a handful of known options.

The distinction becomes important once these decisions accumulate inside an agent run. Imagine that one task requires a model call to plan the work, another to select a tool, another to determine whether a proposed command is risky, another to evaluate the tool result, and another to decide whether the task is complete. Some of those calls genuinely need open-ended reasoning or generation. Others reduce to questions such as:

- Is this command safe to execute automatically?
- Which of these models should handle the request?
- Does this retrieved passage support the claim?
- Did the agent satisfy the requested task?

- Should this result be escalated for review?

The answer space in each case can be defined before inference begins. Yet a general-purpose LLM may still receive the entire state, generate a structured response, and return tokens that the surrounding program immediately converts back into a branch in ordinary code.

When the possible actions are already known, part of an agent's LLM workload may consist of generating representations of decisions that the application will immediately turn back into control flow.

This does not mean that structured outputs are unnecessary. They solve an important interface problem by making generative models easier to integrate into software. The issue is narrower: **structured generation and structured decision-making are still different computational interfaces**. In one case, the model generates a sequence that must conform to a structure. In the other, the model directly evaluates a bounded set of possible decisions.

That distinction is the idea behind **Jev**, released by [TypeSafe AI](#) on September 15, 2026. TypeSafe describes Jev as its first *System One model*: a model designed to receive unstructured state and return typed probabilistic decisions that software can consume directly. Rather than generating the text representing an answer, Jev evaluates questions whose possible output types are declared in advance.

For example, suppose an agent wants to execute:

```
rm -rf ./build
```

A generative model could receive the command and be instructed to return JSON such as:

```
{
  "risk": "low",
  "needs_approval": false
}
```

The application then parses the generated tokens, validates the schema, reads the two values, and selects a branch.

With Jev, the surrounding program instead defines the decisions themselves. One question might classify the action into **low**, **medium**, or **high** risk. Another might ask for the probability that human approval is required. Jev returns typed answers and probabilities, and the application can apply its own policy afterward.

That gives us two architectures for the same operation.

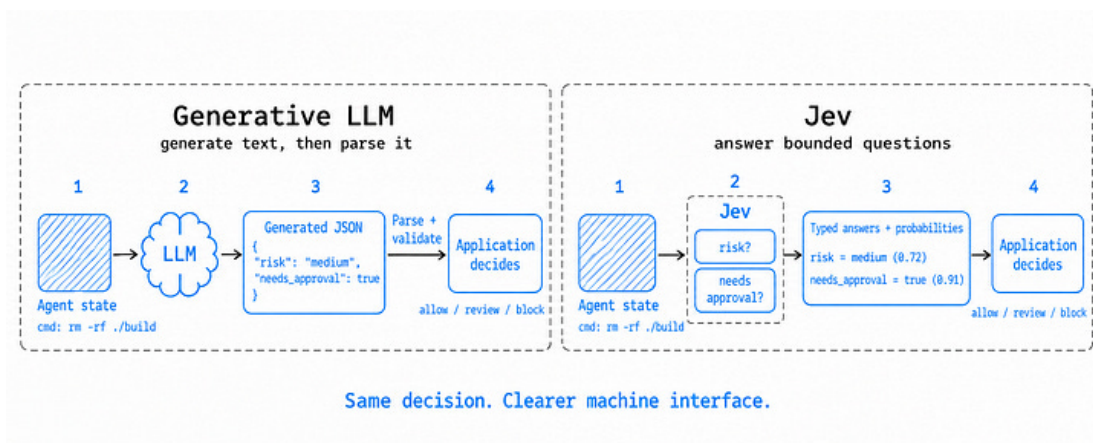


Figure 1—A generative LLM reaches program control flow by producing a constrained sequence that the application parses, while Jev exposes bounded decisions and probabilities directly. In both cases, the surrounding code remains responsible for deciding what action the system is allowed to take.

The architectural difference becomes especially relevant in an agent because these judgments can sit around the expensive generative work

rather than replace it. The main LLM can still inspect an unfamiliar codebase, plan a change, write the patch, or explain a failure.

A decision model can potentially handle narrower calls around that work: routing the original request, checking a proposed action, supervising intermediate results, or determining whether an uncertain case should be escalated.

AI Engineering Courses Bundle at 40%

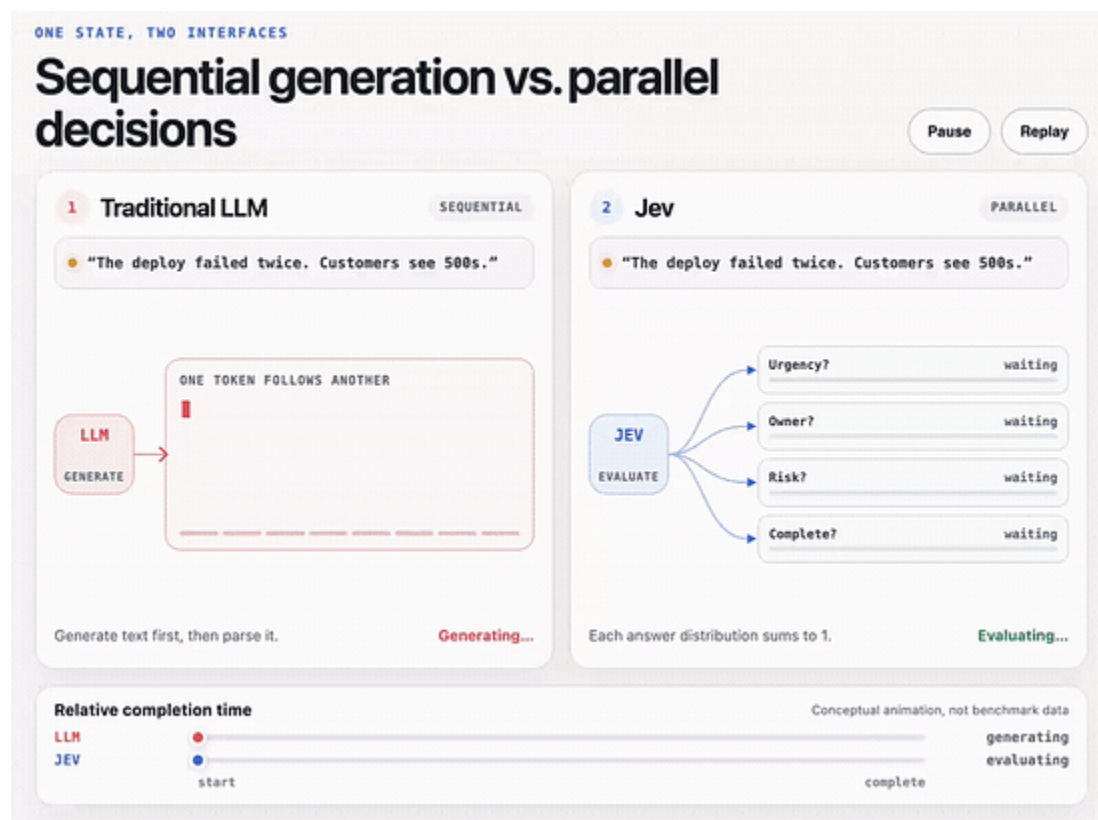


Figure 2. Sequential generation vs. parallel decisions. Given the same agent state, a traditional LLM generates a structured answer token by token, while Jev evaluates multiple bounded questions independently and returns probability distributions for each. The timing shown is a conceptual illustration, not benchmark data.

LangChain has already demonstrated this pattern in its Jev integration. Its middleware can evaluate a proposed tool call before execution and use the result as part of an agent harness, while Jev can also be used for model routing and evaluation. The model therefore becomes one component inside the harness rather than the component responsible for producing the user's final answer.

Whether this separation produces a useful improvement depends on more than TypeSafe's headline latency and cost numbers. The decision model still has to make the correct judgment, expose uncertainty that we can use meaningfully, and fail in ways that the surrounding system can contain. A faster classifier is not useful if its mistakes cause more retries, approvals, or production incidents than the LLM call it replaced.

That is what I want to test in this article. We will build a small decision layer around agent-style tool calls, let Jev classify several aspects of each proposed action, and keep the final allow, review, or block policy in Python. We can then inspect what Jev actually returns, how useful its probabilities are, where it makes mistakes, and how its behavior compares with asking a general-purpose LLM to make the same bounded decisions.

2. How Jev Turns Semantic Judgment into a Software Primitive

AI Books Bundle at a 40% discount

The previous section treated Jev as a different interface for decisions. To understand what that means in practice, we need to look at the unit Jev actually operates on. A request contains a **state** describing the situation and one or more **questions** describing the judgments the application needs from that state.

For our running example, the state could contain the tool call an agent is proposing to execute, together with a small amount of surrounding context:

```
rm -rf ./build
```

The application may want to know several things about this command before allowing it to run. Is the action destructive? How risky is it? Does it require approval? Those judgments are related because they inspect the same command, but they do not necessarily need to be collapsed into one prompt asking the model to decide the complete policy.

TypeSafe exposes three question primitives for expressing these judgments: **Choice**, **Score**, and **Noul**. Each primitive defines the shape of the answer before inference begins, and the returned value is designed to be consumed directly by code.

Choice is appropriate when the answer belongs to a fixed unordered set. For our tool-call gate, we might define the possible risk categories as **low**, **medium**, and **high**. Jev selects one of those options, but it also returns a probability distribution over every option and a confidence value describing how concentrated that distribution is.

A result might therefore look conceptually like this:

```
low: 0.08 · medium: 0.72 · high: 0.20
```

The useful information is not limited to the fact that **medium** won. The surrounding code can see that **high** still carries meaningful probability and apply a stricter policy than it would to a prediction such as **medium: 0.97**.

Score handles ordered judgments. Instead of choosing between unrelated categories, the developer defines a scale whose levels have increasing meaning—for example, **read-only**, **reversible change**, **destructive local change**, and **production-impacting change**. Jev returns a position along that scale, the probability distribution over its levels, and a confidence value. The returned score can fall between defined levels rather than being restricted to one label.

That distinction matters because some agent decisions are naturally ordinal. A command can carry more operational risk than another without belonging to a fundamentally different class. A Score gives the policy layer a quantity that can be thresholded rather than forcing every case into a categorical bucket.

The third primitive, **Noul**, is TypeSafe's Boolean-style question. It answers whether a condition is true by returning a number from 0 to 1 representing the probability of “yes.” Unlike Choice and Score, Noul does not return a separate confidence field because the returned probability itself expresses the uncertainty.

For our command, one Noul question could ask whether the action requires human approval. A result of `0.97` gives the application a strong signal toward approval, while `0.51` indicates that the model is almost evenly split. The policy does not have to treat those two positive answers in the same way.

A Jev answer is more than a label. The probability distribution is part of the interface, which lets the application decide how much uncertainty it is willing to automate.

This also explains why the questions themselves should remain narrow. TypeSafe recommends asking for one focused judgment per question and combining several judgments in code.

A broad instruction such as “analyze this command and determine what the agent should do” moves policy back into the model. Separate questions such as “how risky is this command?” and “does this require human approval?” keep the policy visible in the application.

For the same **rm -rf ./build** state, our decision layer could therefore evaluate three different dimensions at once:

- **Choice—risk category:** low, medium, or high.
- **Score—destructiveness:** a position from read-only through increasingly destructive operations.
- **Noul—needs approval:** the probability that execution should require a person to approve it.

The important architectural detail is that these questions are evaluated independently against the **same state**. The answer to **risk category** is not inserted into the context used by **needs approval**, and the second question does not secretly depend on the first. TypeSafe states that questions sent in the same request are evaluated independently and can be added or removed without changing the other answers.

That allows us to fan several judgments out from one snapshot of agent state and bring the answers back into ordinary code.

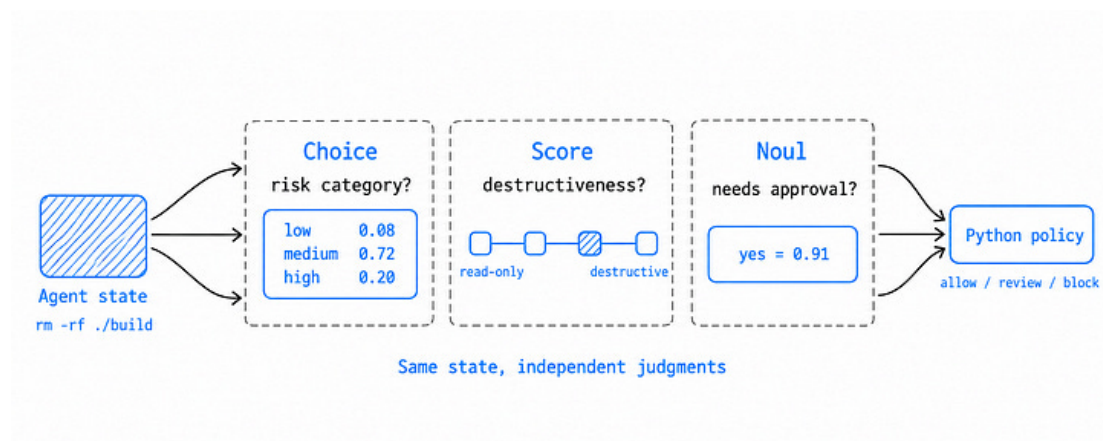


Figure 2—A single snapshot of agent state can fan out into independent Choice, Score, and Noul judgments. Their outputs are combined afterward in application code, so the model evaluates semantics while the program retains control over the policy.

Get 50% off for 1 year